

Лабораторна робота 3: Основи Kubernetes, запуск контейнерів

Завдання 1. Налаштування кластера та перші об'єкти

1.1. Запуск локального кластера та перевірка

Виконані команди:

```
minikube start
kubectl cluster-info
kubectl get nodes -o wide
```

Результати: - Кластер Minikube успішно стартував. - Kubernetes control plane доступний за адресою <https://192.168.49.2:8443>. - У кластері 1 вузол: - minikube, статус Ready, роль control-plane - версія Kubernetes: v1.35.1 - Internal IP: 192.168.49.2

1.2. Збірка локальних образів всередині Minikube Docker environment

Виконані команди:

```
eval $(minikube docker-env)
docker build -t todo-backend ./backend
docker build -t todo-frontend ./frontend
```

Результати: - todo-backend:latest зібрано всередині Docker environment Minikube. - todo-frontend:latest зібрано всередині Docker environment Minikube.

1.3. PostgreSQL: PVC, Deployment, Service

Створені маніфести: - k8s/lab3/postgres-pvc.yaml - k8s/lab3/postgres-deployment.yaml - k8s/lab3/postgres-service.yaml

Виконання:

```
kubectl apply -f k8s/lab3/postgres-pvc.yaml
kubectl get pvc
kubectl get pv

kubectl apply -f k8s/lab3/postgres-deployment.yaml
kubectl apply -f k8s/lab3/postgres-service.yaml
kubectl get pods
kubectl get services
```

Результати: - PVC postgres-pvc отримав статус Bound. - Dynamic provisioning спрацював автоматично, був створений PV. - Pod PostgreSQL перейшов у статус Running. - Service postgres створений як ClusterIP.

1.4. Backend: Deployment, Service

Створені маніфести: - k8s/lab3/backend-deployment.yaml - k8s/lab3/backend-service.yaml

Особливості: - imagePullPolicy: Never - Підключення до БД через DNS-ім'я Service PostgreSQL: postgres - DATABASE_URL=postgresql://todouser:todopass@postgres

Виконані команди:

```
kubectl apply -f k8s/lab3/backend-deployment.yaml
kubectl apply -f k8s/lab3/backend-service.yaml
kubectl wait --for=condition=Available deployment/backend --timeout=180s
kubectl get pods
kubectl get services
kubectl logs deployment/backend --tail=30
```

Результати: - Backend Deployment успішно розгорнуто. - Backend Pod має статус Running. - Service backend створений як ClusterIP. - Backend стартував без помилок і підключився до БД.

1.5. Відповіді на питання

1. Скільки вузлів (nodes) у вашому кластері?
 - 1 вузол.
2. Яку версію Kubernetes використовує Minikube?
 - v1.35.1.
3. Чим відрізняється Pod від контейнера Docker?
 - Docker-контейнер — це один ізольований runtime-екземпляр образу.
 - Pod — це базова одиниця розгортання в Kubernetes, яка може містити один або кілька контейнерів, що ділять мережу, IP-адресу та томи.
4. Навіщо потрібен Service, якщо Pod вже має IP-адресу?
 - IP Pod не є стабільним: Pod може бути пересозданий і отримати нову адресу.
 - Service дає стабільну DNS-точку доступу та балансування між Pod-ами.

5. Що таке декларативний підхід у Kubernetes і чим він відрізняється від імперативного `docker run`?
 - Декларативний підхід: у YAML описується бажаний стан системи, а Kubernetes підтримує цей стан автоматично.
 - `docker run` — імперативний підхід: користувач вручну виконує конкретну команду запуску контейнера.
6. Що таке `dynamic provisioning` у Kubernetes?
 - Це автоматичне створення `PersistentVolume` на основі `PersistentVolumeClaim` через `StorageClass`.
 - У Minikube це працює через вбудований `provisioner` для `storageClassName: standard`.

1.6. Посилання на артефакти в репозиторії

- `k8s/lab3/postgres-pvc.yaml`
- `k8s/lab3/postgres-deployment.yaml`
- `k8s/lab3/postgres-service.yaml`
- `k8s/lab3/backend-deployment.yaml`
- `k8s/lab3/backend-service.yaml`

Завдання 2. Повне розгортання та дослідження

2.1. Frontend: Deployment та Service NodePort

Створені маніфести: - `k8s/lab3/frontend-nginx-config.yaml` - `k8s/lab3/frontend-deployment.yaml` - `k8s/lab3/frontend-service.yaml`

Примітка: - Для frontend використано Service типу NodePort. - Додано ConfigMap з Kubernetes-сумісною nginx-конфігурацією для проксіювання `/api/` на Service backend.

Виконані команди:

```
kubectl apply -f k8s/lab3/frontend-nginx-config.yaml
kubectl apply -f k8s/lab3/frontend-deployment.yaml
kubectl apply -f k8s/lab3/frontend-service.yaml
kubectl wait --for=condition=Available deployment/frontend --timeout=180s
kubectl get nodes -o wide
kubectl get service frontend
```

Результати: - Node IP: 192.168.49.2 - NodePort: 30080 - URL доступу: `http://192.168.49.2:30080`

2.2. Перевірка повної працездатності застосунку

Виконані перевірки:

```

curl -i http://192.168.49.2:30080/api/health
curl -i http://192.168.49.2:30080/api/todos
curl -i -X POST http://192.168.49.2:30080/api/todos \
  -H "Content-Type: application/json" \
  -d '{"title": "k8s-nodeport-todo"}'
curl -i http://192.168.49.2:30080/api/todos
curl -i -X DELETE http://192.168.49.2:30080/api/todos/1
curl -i http://192.168.49.2:30080/api/todos

```

Результати: - GET /api/health -> 200 OK - GET /api/todos -> 200 OK - POST /api/todos -> 201 Created - DELETE /api/todos/1 -> 204 No Content - Застосунок повністю працює через NodePort.

2.3. Результати команд kubectl

Виконані команди:

```

kubectl get all
kubectl describe deployment backend
kubectl get pod backend-7ffd48767c-gbl4p -o yaml

```

Ключові результати: - У кластері працюють 3 Pod-и: postgres, backend, frontend. - Service-и: - postgres -> ClusterIP - backend -> ClusterIP - frontend -> NodePort

2.4. Логи бекенда

Команда для отримання логів:

```

kubectl logs deployment/backend --tail=20

```

Приклад логів:

```

INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
INFO:      10.244.0.5:60538 - "GET /api/health HTTP/1.1" 200 OK
INFO:      10.244.0.5:60564 - "POST /api/todos HTTP/1.1" 201 Created
INFO:      10.244.0.5:60578 - "DELETE /api/todos/1 HTTP/1.1" 204 No Content

```

2.5. Shell всередині Pod бекенда та IP-адреса сервісу БД

Команда для входу в shell Pod бекенда:

```

kubectl exec -it backend-7ffd48767c-gbl4p -- sh

```

Команда для отримання IP сервісу БД зсередини Pod:

```

getent hosts postgres

```

Отриманий результат:

```
10.109.22.61      postgres.default.svc.cluster.local
```

Висновок щодо DNS: - У Docker DNS працював через імена контейнерів у межах user-defined network. - У Kubernetes DNS працює через Service-імена (postgres) та повні DNS-імена виду service.namespace.svc.cluster.local. - Підхід подібний за ідеєю, але в Kubernetes сервісний DNS прив'язаний до Service, а не безпосередньо до Pod/контейнера.

2.6. Видалення Pod бекенду вручну

Виконані команди:

```
kubectl delete pod backend-7ffd48767c-gbl4p  
kubectl get pods -l app=backend -o wide
```

Результати: - Старий Pod: backend-7ffd48767c-gbl4p - Новий Pod: backend-7ffd48767c-h5pdx - Стара IP Pod: 10.244.0.4 - Нова IP Pod: 10.244.0.6 - Час відновлення: приблизно 2 секунди - Service залишився доступним (GET /api/health через NodePort -> 200 OK)

2.7. Відповіді на питання

1. Чи з'явився новий Pod? Скільки часу це зайняло?
 - Так, новий Pod був автоматично створений Deployment-ом.
 - Це зайняло приблизно 2 секунди.
2. Чи змінилось ім'я Pod?
 - Так, ім'я змінилось: з backend-7ffd48767c-gbl4p на backend-7ffd48767c-h5pdx.
3. Чи змінилась IP-адреса Pod?
 - Так, IP змінилась: з 10.244.0.4 на 10.244.0.6.
4. Чи залишився застосунок доступним через Service?
 - Так, застосунок залишився доступним через Service frontend/NodePort.
5. Порівняйте це з лабораторною 2: що відбувалось, коли ви зупиняли контейнер Docker?
 - У Docker при зупинці контейнера сервіс переставав бути доступним, поки контейнер не запускали вручну або не використовувався зовнішній механізм відновлення.
 - У Kubernetes Deployment автоматично пересоздає Pod і повертає застосунок у бажаний стан.

6. Чим ClusterIP відрізняється від NodePort?

- ClusterIP доступний тільки всередині кластера.
- NodePort відкриває сервіс назовні через IP вузла та виділений порт.

2.8. Посилання на артефакти в репозиторії

- k8s/lab3/frontend-nginx-config.yaml
- k8s/lab3/frontend-deployment.yaml
- k8s/lab3/frontend-service.yaml